

Two call centres on one system, calling from the screen on 1 October

What was built on 26–27 September 2026: a cross-tenant leak closed, stepwise onboarding that can give an organisation ConnectDesk alone, ConnectDesk working Fastdial's own CallBox database, the Phase 2 floor on the server, and the calling architecture. That architecture covers click-to-call through the PBX today and a WebRTC phone, with conference, transfer, listen, whisper and barge, once the media edge is installed. It closes with the go-live list for both companies.

27 September 2026 Prepared	Emmanuel Birgen For	Wed 1 October 2026 Pilot starts	connected-suite main 6ba07b0 → 66b3dbf
--------------------------------------	-------------------------------	---	--

01 Where it stands

PIECE	STATE	WHAT THAT MEANS
Cross-tenant leak in ConnectDesk, PeopleHub, Ledgerly	Fixed · live	Every CollectBox read now belongs to the signed-in organisation. Before this, any org holding those systems saw Micromart's book.
Onboarding v2 (company → location → systems → admin → brand)	Live	/onboard . ConnectDesk-only is a valid choice; each system answers on <system>.servicesuitecloud.com/<slug> the moment the org exists.
Fastdial as a ConnectDesk-only organisation	Live	connectdesk.servicesuitecloud.com/fastdial.lms.../fastdial redirects there. Supervisor and Agent roles exist.
ConnectDesk on Fastdial's CallBox database	Live · writes on	Floor, campaigns and builder, work lists, case files, outcomes, promises, callbacks, contracts, agents and phone floor, all on FastDialCallBoxMaster.
Phase 2: the floor on the server	Live	Real seats, logins, lateness, breaks, overruns, pauses, bucket moves and the floor log, for both companies. A live stream feeds every screen.
Calling: desk phone (PBX click-to-call)	Built · test from prod	Call buttons ring the agent's extension through the Yeastar API, then the customer. Proven against the PBX from this workstation; the production test is one click (\$07).
Calling: browser phone (WebRTC)	Built · needs the edge	The phone and the edge configuration are built. It goes live when the edge box is installed (≈45 min, runbook in bridge/edge).
Micromart calling	Needs their PBX	The floor is live. Calls stay simulated until Micromart's PBX type, API and SIP details are in Settings → Connections.

02 The leak, fixed first

Every ConnectDesk page, both desk APIs, PeopleHub and Ledgerly called `collectBoxOrg("micromart")` — a constant — whoever was signed in. Every lender's `Org.systems` was `null`, which means "all systems", and Fastdial's Org Admin holds every right. So Fastdial's administrator, and the staff of Mular, Axe, BuySimu and Techcrast, could open `/desk`, `/people` or `/books` and see Micromart's collections book, staff directory and journal. The desk's write path would also have recorded their outcomes into Micromart's CollectBox, under a fallback agent.

`lib/collectbox/tenant.ts` now resolves CollectBox from the verified session. Only an organisation that owns a CollectBox floor gets one, and everyone else sees the pages' existing "not connected" state. It shipped on its own as `6ba07b0` and deployed successfully.

03 Onboarding, and an address for every system

The wizard is six steps in a landscape split: a rail with the steps and a live identity card on the left, the step on the right.

- **Company:** trading and registered name, what the company does (lender, MFB, SACCO, bank, call centre / BPO), KRA PIN (checked for shape: letter, nine digits, letter), registration number, email, phone, website.
- **Location:** address, town, county (all 47) and a map pin, with latitude and longitude. It uses the Google map where a key is set, and OpenStreetMap otherwise.
- **Systems:** any combination of the seven, with presets ("Call centre" = ConnectDesk alone). The organisation's address is checked live as it is typed, with every system's resulting link shown.
- **Administrator, Branding** (logo and colours on every door and email), then **Review**.

`POST /api/orgs` stores the profile, an explicit `Org.systems` list (never null) and starter roles that follow the systems. A ConnectDesk organisation gets *Call-centre Supervisor* and *Agent*, using two new rights, `desk.supervise` and `desk.dial`.

No deployment is involved in creating an organisation's address. The door at `/<slug>` answers on every host and wears the org's logo. A door the org does not hold sends visitors to one it does: Fastdial typing `lms.../fastdial` lands on ConnectDesk. `/platform` has an *Onboard an organisation* button.

04 One ConnectDesk, three kinds of floor

ConnectDesk asks, for the signed-in organisation, where its book lives (`lib/desk/source.ts`), and renders the matching screens and menu:

KIND	EXAMPLE	BOOK	BUCKETS
collectbox	Micromart	CollectBox on its ServiceSuite SQL Server, through the bridge	The seven arrears bands
callbox	Fastdial	CallBox Postgres (vault: <code>CALLCENTRE</code>)	Open campaigns
none	A new call centre	Nothing yet. The floor, settings and phone work; the book waits for Settings → Connections	—

Fastdial's CallBox, as it actually works

Read from the code (`C:\GIT\CallBox`) and the live schema, read-only, on 26 September:

- **companies** are the lenders Fastdial collects for: 15, including KCB Digital and Conventional, 4G, Zenka, Pi Capital (Bayes), Kingdom Bank and Mular Credit.
- **contracts** (50) are uploaded batches. **contractdata** (151,789) has one debtor per row, with `olb` , arrears, `assignedagent` , `calls_made` , `actionstatus` (the last outcome) and `campaignid` .
- A **campaign** is a filter over one contract for a set of agents: balance range, arrears range, calls made, last outcomes, worked before or not, paid something or not, and last-worked dates. Creating it *locks* every matching row that is in the pool and assigned to those agents. Saving an outcome puts the row back in the pool (`campaignid = 0` , which means available, not closed), and the campaign closes itself when empty.
- **Outcomes** come from `paymentresponse` (21 of them). Some schedule a promise (a `promisedtopay` row) and some a callback (`task_scheduler`).
- The PBX's own event receiver on `213.148.17.228:8260` writes `callrings` and `calldr` live (3,229 CDRs in the last 24 h when checked), keyed by the Yeastar call id.

Same statements, side by side

ConnectDesk writes with the statements CallBox itself runs, so the two can work the same floor through the pilot and each is true of the other's work. Every write path ran against the live schema inside a transaction that was then rolled back. The record was unchanged afterwards and no campaign row was added.

CONNECTDESK	MIRRORS CALLBOX	PROVEN
<code>logCallStart</code>	<code>HomeController.SaveCallLog</code>	Call-log row with the PBX call id
<code>saveDisposition</code> (campaign)	<code>CallsController.PostCallResponse</code>	Outcome, promise, callback; row back to the pool; campaign auto-close
<code>saveDisposition</code> (one-off)	<code>CallsController.PostSearchResponse</code>	The customer stays in their campaign
<code>createCampaign</code>	<code>CallsController.SaveCallCampaign</code>	Locked 908 rows; the live preview counted 908
<code>closeCampaign</code>	<code>CallsController.CloseCampaign</code>	Released 1,030 rows (rolled back)

Two deliberate improvements: an outcome's rules (a promise needs an amount and a date, a callback needs a time) are enforced on the server, and the follow-up task is written inside the transaction, where CallBox writes it on a separate connection. CallBox's timestamps are Nairobi wall time; ConnectDesk reads them as +03:00, so nothing shifts three hours on Vercel.

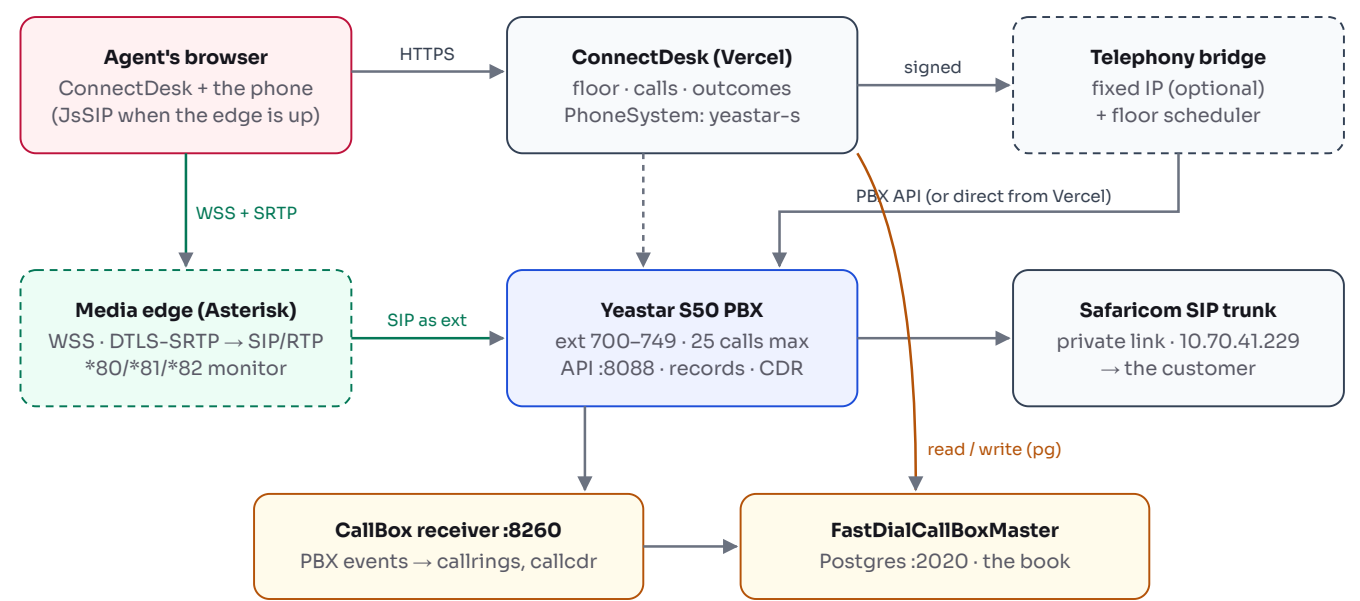
05 Phase 2 — the floor on the server

The Phase 1 engine's pure rules are still the rules: `apply` for actions, plus a new `realTick` for time. The tick runs the schedule's 08:00 start and 17:30 stop on working days, the overrun alert, pause expiry and wrap-up to ready, with none of the simulated calls. They now run on the server (`lib/desk/floor/server.ts`) over:

TABLE	HOLDS
DeskFloor	The master switch, break types, schedule, dialling rules and bucket routing, and the once-a-day schedule flags
DeskSeat	One row per seat: login, extension, agent id in the org's own system, bucket, mode, and live state
DeskSeatSession	Every login and logout, with lateness in seconds (the adherence report's evidence)
DeskBreakLog	Every break with its allowance, overage and who recalled it
DeskFloorEvent	The floor log, append-only
DeskCall · DeskPbxSession	Every call placed through ConnectDesk, tied to the PBX call id; the shared PBX API token

- **Routes:** `GET/POST /api/desk/floor` (an agent opening ConnectDesk logs their seat in), `/api/desk/floor/stream` (server-sent events) and `/api/cron/desk-floor` (the scheduler for when no screen is open). Only changed rows are written, so actions on different seats never contend.
- **Permissions:** an agent can act only on their own seat. A supervisor can act on any seat, and in "View as" the log records the action as that agent's.
- **The roster seats itself** from the org's own system: 26 working agents from CollectBox for Micromart, and 31 from CallBox for Fastdial, with their campaigns as buckets. Importing agents (Settings → Seats) gives each one a login with the Agent role and a one-time password.
- **Verified end to end** on Fastdial: start floor → log in (late by the roster) → washroom → recall → move two seats to a campaign → pause → log out. It produced one session with lateness, one recalled break and nine log events. The test rows were then removed.

06 Phase 3 — how calls work



Solid: works on 1 October with nothing new installed. Dashed: the media edge (browser phone) and the bridge (only if the PBX's API refuses Vercel's addresses).

One pipeline, two ways to hear the call

Every call goes through `lib/desk/calls.ts`. It gets a `DeskCall` row, a `CallBox` call-log row carrying the PBX call id (exactly what `CallBox` writes), and a live state the phone and the case file follow: dialling → ringing → answered → ended.

DESK PHONE · DAY ONE

PBX click-to-call

The PBX rings the agent's own handset or softphone, auto-answers it and dials the customer: Yeastar `dial_outbound`, the mechanism `CallBox` already uses thousands of times a day. Hold, mute, transfer and hang-up go through the PBX API. The state comes from the PBX's live call list, cached two seconds per org (30 agents cost the PBX one request, not 30), and then from the CDR.

BROWSER PHONE · WITH THE EDGE

WebRTC as the extension

The browser registers through the edge as the agent's extension, and calls are SIP from the tab. The engine prefers G.711 so the edge never transcodes. The PBX still sees extension 715: its CDR, recording, outbound rights and monitor permissions are unchanged.

In-call capabilities

CAPABILITY	DESK PHONE	BROWSER PHONE	HOW
Click-to-call from any row, case file, promise or callback	Yes	Yes	One <code>desk:dial</code> event; the phone does the rest
Hold · resume · mute	Yes	Yes	PBX API / SIP re-INVITE
Keypad tones (IVRs, bank menus)	—	Yes	RFC 4733 from the browser
Blind transfer	Yes	Yes	PBX <code>calltransfer</code> / SIP REFER

Add someone · three-way conference	—	Yes	The second call is dialled while the first holds, then Merge mixes it in the tab: each party hears the agent and the other
Supervisor listen · whisper · barge	Yes	Yes	Seat drawer → PBX monitor API rings the supervisor's phone. On the edge, <code>*80/*81/*82+ext</code> works on any PBX
Outcome written to the call's own log row	Yes	Yes	The phone's wrap-up or the case file; the CallBox row carries the PBX call id
Recording	PBX	PBX + edge	The PBX records as today; the edge can also record (a setting). Playback in ConnectDesk is not built yet.
Predictive / progressive dialling	Phase 4	Phase 4	The floor has the modes, pacing and rules; the engine that dials ahead is next

The PBX facts this rests on (review of 20 August)

Yeastar S50, firmware 30.15. It holds **50 of 50** extensions (700–749), so no new extension can be added without freeing one, and 23 placed no calls that day. Its ceiling is **25 concurrent calls** (peak measured: 18). The trunk is SAFARICOM-SIP over a private link, with FOX1 configured but idle. The S-Series has **no WebRTC**, which is why the edge exists. The API gives click-to-call, hold, mute, transfer, listen, whisper, barge and live call lists. ConnectDesk holds **one shared API token**, where CallBox logged in before every action (5,768 logins in one day). It also logs in with **exactly the report address CallBox's receiver uses**, because a Yeastar login re-points the PBX's live event feed.

07 Go-live — Fastdial

#	STEP	WHO	NOTES
1	Sign in at <code>connectdesk.servicesuitecloud.com/fastdial</code> as Fastdial's admin, then Settings → Connections → Test the PBX from this server	You	"Logged in. 50 extensions..." means Vercel can drive the PBX. Error 20015 (IP refused) means step 2.
2	Only if step 1 said 20015: run the telephony bridge on <code>servicesuite-in (bridge/telephony-bridge)</code> , set its URL and secret in the phone settings, and add 213.148.17.228 to the Yeastar API's allowed IPs	You	≈20 min. The bridge also runs the floor's minute scheduler, which is worth having either way.
3	Settings → Seats → Bring agents in from CallBox: select the agents and hand each their one-time password	You / Fastdial	Each gets a login (Agent role), a seat and their extension from CallBox. There is no SMS sender on Fastdial yet, so sign-in is password only. It is audited, and adding an SMS sender turns on the daily code.
4	An agent signs in, opens a campaign, presses Call : their desk phone rings, then the customer. Save the outcome	You at Fastdial	This is the day-one proof. Check that CallBox shows the same outcome on the same row.
5	Supervisor: Command centre → start the floor → a seat → Listen	You	Needs the supervisor's own extension on their seat and monitor permissions on the Yeastar (edge runbook, "On the Yeastar").
6	Browser phone: install the edge on a small Ubuntu box on Fastdial's LAN (<code>bridge/edge/install.sh</code>), set its address and each seat's SIP password in ConnectDesk	You	≈45 min plus DNS. Log MicroSIP or Linkus out on those PCs first, because an extension holds one registration.
7	Point <code>CRON_SECRET</code> for the bridge (or any 1-minute scheduler) at <code>/api/cron/desk-floor</code>	You	The live stream ticks the floor while screens are open; this covers the rest.

08 Go-live — Micromart

The live floor is already Micromart's: 26 CollectBox agents seated, the seven bands with real counts, supervisors on the Command Centre. What calling needs from Micromart:

- **What the PBX is** (make, model, firmware) and its admin/API access. If it is a Yeastar S-Series, the driver exists. If it is Asterisk/FreePBX or 3CX, a driver slots in behind the same `PhoneSystem` interface, and the edge works with any SIP PBX regardless.
- **Each agent's extension and its SIP password**, entered on the seats.
- Their CollectBox `calldr` stopped being written in September 2023, so ConnectDesk's own call ledger (DeskCall) will be the call record until their PBX's events are connected.

09 Risks, and what is not done

ITEM		RECOMMENDATION
CallBox is connected as the Postgres superuser (the same credential sits in plain text in CallBox's appsettings.json, for a port that answers from the internet)	Risk	Create a connectdesk login with rights on the CallBox tables only, and replace the string in Settings → Connections (tested before it saves). Rotate the superuser password and restrict port 2020.
Writes to CallBox are on for Fastdial	Decision	Needed for the pilot. One switch in Settings → Connections makes the floor read-only.
The PBX is full: 50 of 50 extensions	Decision	Reclaim the idle ones before adding anyone.
Recording and transcription	Legal	Customers must be told calls are recorded; recordings are personal data under the Data Protection Act 2019. Agree retention and access with both companies before the edge records anything.
Not built yet	Next	Predictive dialling (Phase 4). Live transcription and the AI co-pilot on real calls (Phase 5; the phone shows no scripted transcript on a real call). Recording playback in the case file. A driver for Micromart's PBX once it is known.

10 Where it lives

PATH (CONNECTED-SUITE)	ROLE
<code>lib/collectbox/tenant.ts</code>	The tenant gate on every CollectBox read
<code>components/onboard/*</code> , <code>app/api/orgs/*</code> , <code>lib/suite/links.ts</code>	Onboarding v2 and every system's address
<code>lib/desk/source.ts</code> , <code>lib/desk/nav.ts</code>	Which floor an org has, and its menu
<code>lib/callbox/{client,floor,write}.ts</code> , <code>components/desk/callbox/*</code>	CallBox: connection, reads, CallBox-exact writes, screens
<code>lib/desk/floor/server.ts</code> , <code>app/api/desk/floor/*</code>	Phase 2: the floor on the server, the stream, the scheduler
<code>lib/desk/telephony/*</code> , <code>lib/desk/calls.ts</code>	The PhoneSystem interface, the Yeastar S driver, the call pipeline
<code>components/desk/phone/{useCallEngine,RealCall}.tsx</code>	The phone's engine: desk phone, WebRTC, the in-tab conference
<code>lib/desk/{settings,seats,edge}.ts</code> , <code>components/desk/settings/*</code>	Connections, seats and agent import, the generated edge config
<code>bridge/edge/*</code> , <code>bridge/telephony-bridge/*</code>	The media edge kit and runbook; the fixed-IP bridge
<code>sql/connectdesk.sql</code>	The schema delta (applied): Org profile, vault kinds, the seven Desk tables